

Comparing LoRA, Adapters, and BatchNorm Tuning for Food-101 Classification

By: Peter Ngyuen, Inaara Rajwani, Austin Coulter

Summary: This project investigates how parameter-efficient fine-tuning strategies affect both classification accuracy and robustness for food image recognition on the Food-101 dataset (Bossard et al., 2014). Fully retraining large pretrained models is often computationally expensive, motivating the use of lightweight adaptation methods that update only a small subset of parameters. We evaluate two pretrained convolutional backbones, ResNet18 (He et al., 2016) and EfficientNetV2-S (Tan & Le, 2021), using linear probing as a baseline and comparing three parameter-efficient fine-tuning (PEFT) approaches: Low-Rank Adaptation (LoRA; Hu et al., 2022), task-specific adapters (Houlsby et al., 2019), and BatchNorm tuning (Ioffe & Szegedy, 2015). In addition, we propose a custom ResNet18-based model combining CBAM attention (Woo et al., 2018), adapters, BatchNorm tuning, and stronger data augmentation. Models are evaluated on the clean Food-101 test set and multiple corrupted variants to assess robustness. Results show that all PEFT methods substantially outperform linear probing. EfficientNetV2-S with BatchNorm tuning achieves the highest clean accuracy (86.47%) and the best overall robustness, while the proposed custom ResNet18 model yields the strongest ResNet-based performance and improved robustness. These findings highlight meaningful trade-offs among accuracy, speed, and parameter efficiency, demonstrating that selective fine-tuning can be highly effective for this task.

1. Introduction

Recent advances in deep learning have produced powerful convolutional neural networks that can be adapted to many downstream tasks using pretrained representations. This study focuses on two widely used architectures for image classification: ResNet and EfficientNetV2.

ResNet, introduced by He et al. (2016), addresses the degradation problem in deep networks through residual connections that add the input of a layer to its output. This design improves gradient flow and enables the successful training of substantially deeper models, mitigating vanishing gradient issues. In contrast, EfficientNetV2, proposed by Tan and Le (2021), emphasizes both accuracy and computational efficiency through architectural refinements and compound scaling, which jointly balance depth, width, and input resolution.

Although these backbones are readily available as pretrained models, adapting them to new tasks remains challenging. Training such networks from scratch is computationally expensive and often impractical. To address this limitation, parameter-efficient fine-tuning (PEFT) methods have emerged as effective alternatives (Houlsby et al., 2019; Hu et al., 2022). These approaches modify only a small subset of parameters while keeping most of the pretrained network frozen.

Among PEFT techniques, Low-Rank Adaptation (LoRA) introduces trainable low-rank matrices into existing weight layers, enabling efficient updates without changing the original weights (Hu et al., 2022). Task-specific adapters insert small bottleneck modules within the network that are trained for each task while the backbone remains fixed (Houlsby et al., 2019). BatchNorm tuning adapts pretrained networks by updating only the Batch Normalization scale and shift parameters, allowing efficient distribution alignment with minimal parameter updates (Ioffe & Szegedy, 2015).

In this project, we study how these fine-tuning strategies affect performance and robustness on the Food-101 dataset (Bossard et al., 2014). We compare linear probing, multiple PEFT methods, and a custom architecture that integrates BatchNorm tuning, task-specific adapters, and Convolutional Block Attention Modules (CBAM), which introduce both channel-wise and spatial attention (Woo et al., 2018).

Across our experiments, all PEFT methods substantially improve performance compared to linear probing, with gains of 10–20 percentage points in mean accuracy. BatchNorm tuning consistently yields the strongest overall results despite using fewer trainable parameters than LoRA. EfficientNetV2-S models outperform ResNet18 variants in accuracy but incur significantly higher inference costs, revealing a clear trade-off between accuracy and efficiency.

2. Methodology

2.1 Problem Setup and Models

We evaluate image classification performance on the Food-101 dataset, which contains 101 food categories with substantial intra-class variation and fine-grained visual differences

(Bossard et al., 2014). Experiments are conducted using two pretrained convolutional backbones: ResNet18 (He et al., 2016) and EfficientNetV2-S (Tan & Le, 2021). The final classification layer of each model is adapted to the Food-101 label space.

Our evaluation focuses on classification accuracy, robustness under distribution shift, inference speed, and the number of trainable parameters. We compare linear probing with several PEFT strategies: LoRA, task-specific adapters, BatchNorm tuning, and a custom attention-augmented model.

2.2 Fine-Tuning Methods

2.2.1 Linear Probing

In linear probing, all backbone parameters are frozen, and only the final classification layer is trained. This serves as a baseline to evaluate how well pretrained representations transfer to the target task without modifying internal feature extractors. All fine-tuning methods described below build upon this setup by additionally introducing trainable components while keeping the backbone frozen.

2.2.2 LoRA

LoRA introduces trainable low-rank matrices that approximate updates to selected convolutional weights while keeping original parameters frozen (Hu et al., 2022). We apply LoRA to selected convolutional layers using the LoRAConv2d implementation. Only LoRA parameters are updated during training, resulting in a parameter-efficient adaptation.

2.2.3 Adapters

Adapter modules are lightweight trainable components inserted into intermediate layers of the network (Houlsby et al., 2019). Each adapter follows a bottleneck design, consisting of a down-projection to a lower-dimensional representation followed by an up-projection back to the original feature dimension.

For the ResNet architecture, adapters are inserted after each of the four main stages (layers 1–4) with a reduction factor of 8. For EfficientNet, adapters also use a reduction factor of 8 but are only introduced after selected stages, specifically stages 3, 5, and 6. The remaining backbone network remains frozen, and only the adapter parameters are updated.

2.2.4 BatchNorm Tuning

BatchNorm tuning updates only the scale and shift parameters of Batch Normalization layers while keeping all other weights frozen (Ioffe & Szegedy, 2015). This approach allows the network to realign feature distributions to the target domain with minimal parameter updates.

2.2.5 Custom Method (CBAM + Hybrid Tuning)

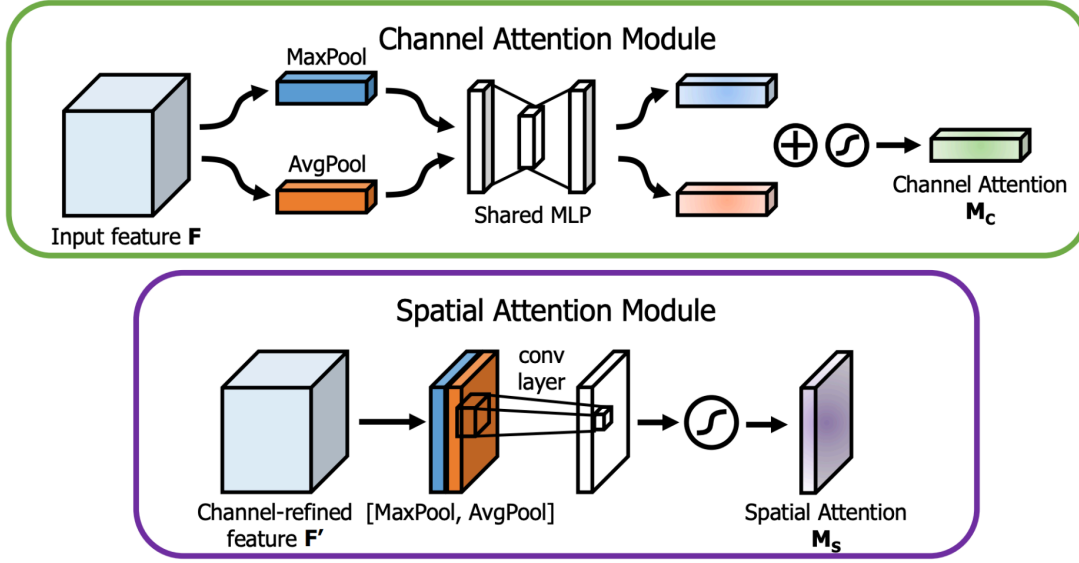


Figure 1: Diagram of CBAM module

Our proposed method extends ResNet18 by incorporating Convolutional Block Attention Modules (CBAM) (Woo et al., 2018), which apply both channel-wise and spatial attention to enhance feature representations.

Each CBAM block operates as follows. First, the input features pass through a task-specific adapter consisting of a bottleneck structure as described above. The adapted features are then refined through channel and spatial attention mechanisms.

The channel attention module emphasizes informative feature channels. It is computed by applying global average pooling and max pooling across spatial dimensions, followed by a shared multilayer perceptron (MLP) that produces channel-wise attention weights per channel. These weights rescale the feature maps, amplifying relevant channels and suppressing less informative ones.

The spatial attention module focuses on important spatial regions. It is computed by applying average pooling and max pooling across the channel dimension, producing a $2 \times H \times W$ feature map. This is followed by a convolutional layer with a 7×7 kernel to generate a spatial attention map, which is multiplied element-wise with the input feature map.

Finally, the attention-refined features are combined with the original input via a residual connection. In addition, we combine BatchNorm tuning with task-specific adapters to further improve adaptability. This hybrid approach aims to balance parameter efficiency with increased representational power.

2.3 Training Setup

All models are trained on the Food-101 dataset using standard preprocessing and data augmentation. For all fine-tuning methods, we apply a consistent set of basic transformations, including resizing and normalization. For the proposed custom model, we additionally incorporate stronger data augmentation techniques, including blur, rotation, color perturbations, and random erasing, to improve robustness and generalization.

We use cross-entropy loss for optimization. Training is performed using the Adam optimizer with a learning rate of 10^{-3} , and all models are trained for the same number of epochs to ensure a fair comparison.

2.4 Evaluation Protocol

Model performance is evaluated using classification accuracy on multiple modified test sets designed to assess robustness under distribution shifts, including clean, blurred, rotated, downsampled, and masked inputs. We also measure the number of trainable parameters for each method to assess parameter efficiency. Inference time is recorded to evaluate computational cost and deployment feasibility. These metrics are used to compare trade-offs between accuracy, efficiency, and model complexity.

2.5 Implementation Details

All experiments are implemented in PyTorch (Paszke et al., 2019). Pretrained weights are obtained from standard model libraries. Training and evaluation are conducted using a consistent pipeline across all methods to ensure reproducibility and a fair comparison. Models are trained until performance plateaus, which typically occurs around 7–8 epochs.

3. Results

3.1 Comparing performances across models

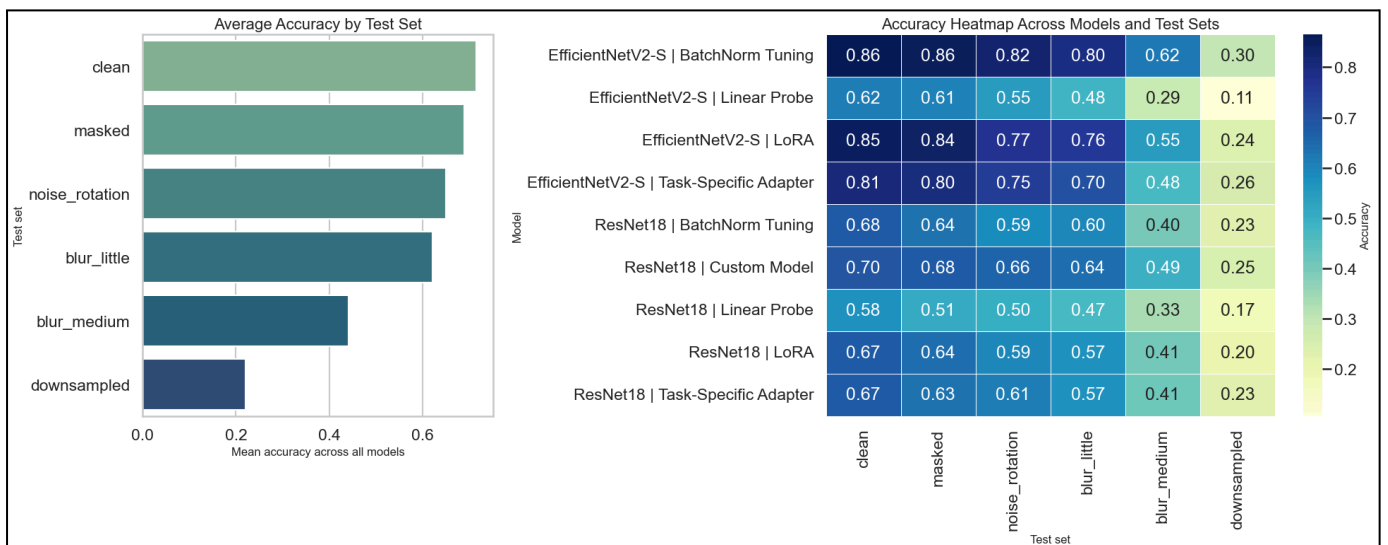


Figure 2: Accuracy across test sets and models.

Table 1 summarizes the main quantitative findings from the final models.

Backbone	Method	Mean Accuracy
ResNet18	Linear probe	42.69%

ResNet18	BatchNorm tuning	52.10%
ResNet18	Task-specific adapter	52.18%
ResNet18	LoRA	51.36%
ResNet18	Custom model	57.15%
EfficientNetV2-S	Linear probe	44.15%
EfficientNetV2-S	BatchNorm tuning	70.95%
EfficientNetV2-S	Task-specific adapter	63.04%
EfficientNetV2-S	LoRA	66.77%

Table 1a. Accuracy and robustness results

Backbone	Method	Trainable Parameters	Throughput (images/s)	Avg Time/ Image
ResNet18	Linear probing	51,813	19,428	0.051 ms
ResNet18	BatchNorm tuning	51,813	18,914	0.054 ms
ResNet18	Task-specific adapter	139,933	16,556	0.060 ms
ResNet18	LoRA	18,576	18,576	0.054 ms
ResNet18	Custom Model	194,469	9,786	0.102 ms
EfficientNet V2-S	Linear Probing	129,381	359	2.878 ms
EfficientNet V2-S	BatchNorm Tuning	283,253	348	2.786 ms
EfficientNet V2-S	Task-Specific Adapter	153,729	357	2.805 ms
EfficientNet V2-S	LoRA	1,511,997	472	2.120 ms

Table 1b. Model efficiency comparison

EfficientNetV2-S consistently outperformed ResNet18 across all methods. Averaged over all variants, EfficientNetV2-S achieved 61.23% mean accuracy compared to 51.10% for ResNet18, with this gap appearing on both clean and corrupted test sets.

The superior performance of EfficientNetV2-S can be attributed to its stronger pretrained representation and more effective multi-scale feature extraction(Tan & Le, 2021). Its architecture is built on Mobile Inverted Bottleneck Convolution (MBConv) blocks, which expand channel dimensions using 1×1 convolutions, followed by depthwise convolutions to learn spatial features, and then project back to a lower dimension. This structure enables the model to capture both fine-grained local textures and higher-level semantic features more effectively than standard ResNet blocks. In the Food101 dataset, where classes often differ by subtle cues such as texture, garnish, plating style, and small structural details, preserving these distinctions is critical.

In addition, EfficientNetV2 uses compound scaling, which jointly increases depth, width, and input resolution in a balanced manner. This enables the model to better utilize higher-resolution inputs and build richer feature hierarchies. The inclusion of squeeze-and-excitation (Hu et al., 2018) attention further improves performance by adaptively reweighting channel importance, helping the model focus on the most informative features. This is particularly beneficial under corruptions such as masking and blur, where robustness depends on emphasizing meaningful signals.

However, these gains come at a significant efficiency cost. ResNet18 processed approximately 16,652 images per second on average, while EfficientNetV2-S processed only about 384 images per second, making ResNet18 around 50 times faster at inference. This highlights a key trade-off: EfficientNetV2-S is preferable when maximizing accuracy is the priority, particularly for fine-grained or robustness-critical tasks, whereas ResNet18 remains a strong choice for latency-sensitive or resource-constrained settings.

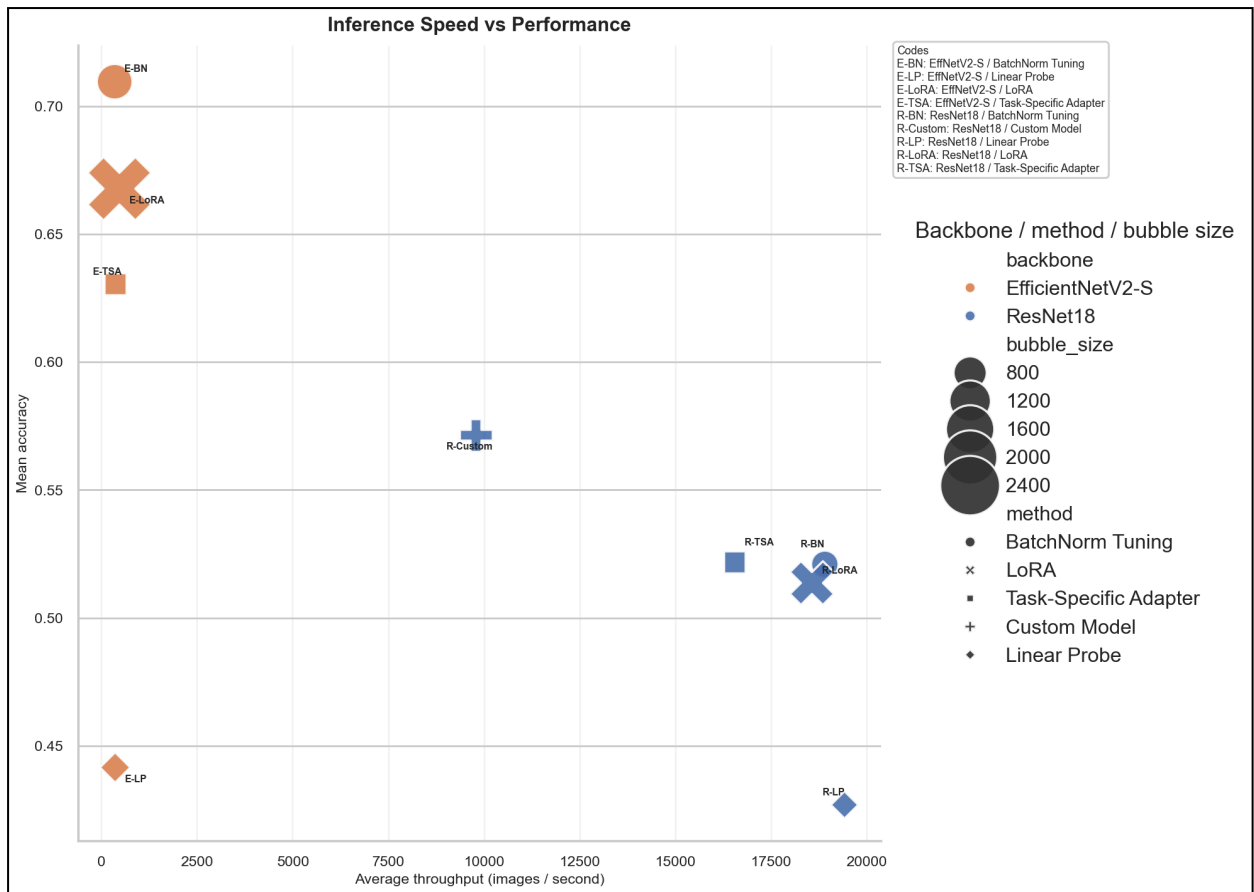


Figure 3: Inference Speed vs Performance Across Models

3.2 Comparing different fine-tuning techniques

Linear probing was the weakest baseline for both backbones, with ResNet-18 achieving 42.69% mean accuracy and EfficientNetV2-S reaching 44.15%. Despite this lower performance, these results remain informative because they isolate the contribution of the pretrained feature extractor. Given the fine-grained nature of many classes, a frozen backbone with only a classifier head struggles to capture subtle visual differences. In contrast, all additional tuning strategies led to substantial improvements over linear probing, indicating that the added trainable components enabled meaningful task-specific adaptation rather than simply increasing model complexity.

BatchNorm tuning was the strongest overall PEFT strategy in this project, particularly for EfficientNetV2-S. Compared to the EfficientNet linear probing baseline, it improved mean accuracy by 26.8 percentage points, from 44.15% to 70.95%, while using only 283,253 trainable parameters. It also ranked first on every individual test variant and outperformed LoRA fine-tuning despite using far fewer parameters, demonstrating strong parameter efficiency. This result highlights the effectiveness of targeted parameter updates. Rather than introducing large numbers of new parameters, BatchNorm tuning adjusts feature scaling and normalization statistics throughout the network. When the backbone is already well pretrained, these updates can align internal representations with the Food101 domain without disrupting useful general features learned from ImageNet. A similar pattern is observed with ResNet18, although the gains are smaller. BatchNorm tuning improved mean

accuracy from 42.69% to 52.10%, confirming that this approach is effective across different backbone architectures.

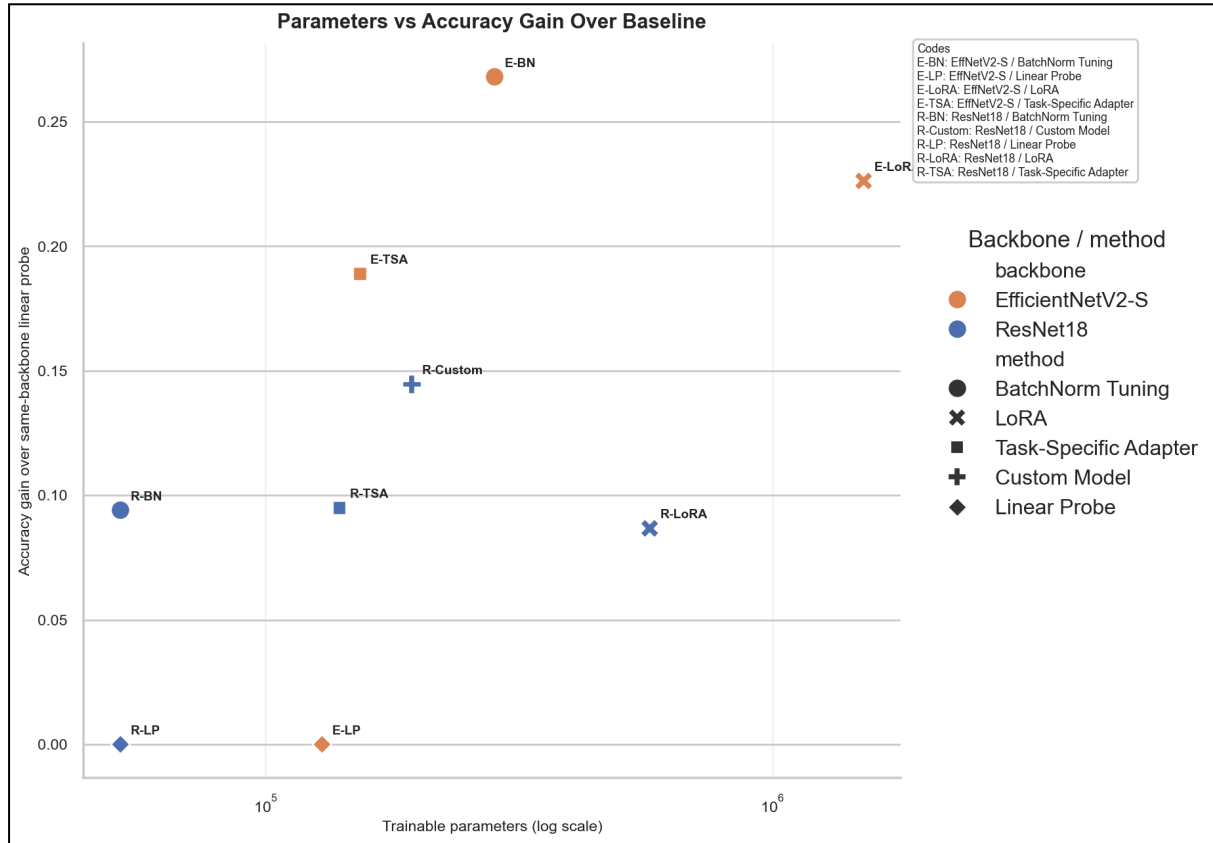


Figure 4: Improvement in performance relative to baseline models as a function of trainable parameter count

Task-specific adapters were efficient and reliable across both backbones. The EfficientNetV2-S adapter model reached 63.04% mean accuracy using only 153,729 trainable parameters. This is notable because it required fewer trainable parameters than EfficientNet BatchNorm tuning, yet still achieved a large improvement over the linear baseline. The same pattern holds for ResNet18. The adapter model improved the mean accuracy to 52.18%, which is slightly above BatchNorm tuning and better than LoRA. This suggests that small residual bottleneck modules inserted in the feature hierarchy are a practical way to let the model specialize to the task while preserving the pretrained backbone. From a design point of view, adapters are appealing because they are modular and easy to interpret: they add task-specific capacity exactly where needed, without retraining the full model. In this project, they provided a good compromise between performance and parameter count.

LoRA improved performance, but it was not the best trade-off here. LoRA improved over the linear baselines for both backbones, but it did not become the top method in this set of experiments. On EfficientNetV2-S, LoRA reached 66.77% mean accuracy, which is strong,

but still below BatchNorm tuning. On ResNet18, LoRA reached 51.36%, which is also below the custom model and slightly below the task-specific adapter result. The most interesting detail is the efficiency comparison. The EfficientNet LoRA model used 1,511,997 trainable parameters, far more than either BatchNorm tuning or the task-specific adapters, but it still did not achieve the highest accuracy. This tells us that increasing trainable parameter count alone does not guarantee better results. The placement of those parameters matters just as much as the amount. A likely reason is that the chosen LoRA rank and insertion points were not fully optimized for these convolutional backbones. LoRA is powerful, but its performance can be sensitive to where the low-rank updates are applied and how aggressively the rank is tuned. In this project, it behaved like a strong middle-ground method, but not the best one.

Among the ResNet18 models, the custom model achieved the best performance, with 57.15% mean accuracy and 69.56% accuracy on the clean test set. Compared to the strongest standard ResNet18 PEFT baseline (e.g., LoRA, BatchNorm tuning, or task-specific adapters used individually), this represents an improvement of roughly 5 percentage points in mean accuracy. The model was designed to go beyond standard PEFT methods by combining CBAM-style attention modules, BatchNorm tuning, and stronger data augmentation. CBAM extends squeeze-and-excitation by adding a spatial attention mechanism, allowing the network to emphasize both important channels and spatial regions. This combination not only improved accuracy but also increased robustness. The model exhibited the smallest drop from clean to corrupted performance, with accuracy decreasing from 69.56% to 54.66%, a drop of 14.9 percentage points. These results suggest that jointly enhancing feature representation and adaptation mechanisms is more effective than applying a single technique in isolation, leading to both improved performance and greater robustness to distribution shifts.

Overall, as shown in Figure 4, there is a general trend that increasing the number of trainable parameters leads to higher accuracy. However, LoRA does not follow this pattern. This suggests that both the number of trainable parameters and where they are introduced in the model are equally important for performance.

3.3 Impact of Test Modifications on Accuracy

The hardest setting by a very large margin was downsampled, which caused an average drop of about 49.5 percentage points relative to the clean test set. This tells us that the models depend heavily on fine visual detail. Food images often differ in texture, edge shape, garnish placement, or small ingredients. When spatial resolution is reduced, that information disappears quickly, so the classification problem becomes much harder. By contrast, the masked set remained relatively manageable. The average accuracy dropped by only about 2.6 points from the clean set. This suggests that the models can often still identify the food category from partial evidence, such as shape, color, or context remaining in the visible part of the image. Noise and rotation also reduced performance, but not as severely as strong blur or downsampling. Another useful observation is the difference between the two blur settings. The average model score fell from 62.04% on blur_little to 44.12% on blur_medium. This gap shows that the models tolerate mild blur reasonably well, but become much less reliable when the blur removes mid-level texture cues.

If the goal is the highest accuracy and strongest robustness, the best choice is EfficientNetV2-S with BatchNorm tuning. It dominated the accuracy on every dataset and achieved the best clean and corrupted performance. If the goal is fast inference on limited hardware, the ResNet18 family is much more practical. Even the custom ResNet18 model, which is slower than the other ResNet variants, still runs far faster than the EfficientNet models. This makes ResNet18 attractive for real-time or edge settings. If the goal is a strong balance between parameter efficiency and performance, BatchNorm tuning and task-specific adapters stand out more than LoRA in these experiments. They delivered large gains over the linear baseline without requiring a very large trainable parameter budget.

3.4 Discussion of Design Choices

Several design choices influenced the final results. First, using ImageNet-pretrained weights was essential. Without those pretrained representations, the parameter-efficient strategies would have much less to build on. Second, the use of multiple corrupted test folders made the comparison much more meaningful than a clean-only evaluation. It exposed which methods truly generalized and which ones only fit the original image distribution.

Third, the custom model shows that architectural changes should be combined with training strategy choices rather than treated separately. Attention, augmentation, and selective fine-tuning worked well together. This combination improved both accuracy and robustness, which is more valuable than optimizing for just one metric.

Finally, the results reinforce a broader lesson in transfer learning: the most effective method is not always the one with the most trainable parameters. Careful placement of a small number of updates can outperform a larger but less targeted adaptation strategy.

4. Model Interpretation using Grad-CAM

Grad-CAM Visualization Results

The Grad-CAM visualizations reveal clear differences in attention behavior across models and fine-tuning strategies.

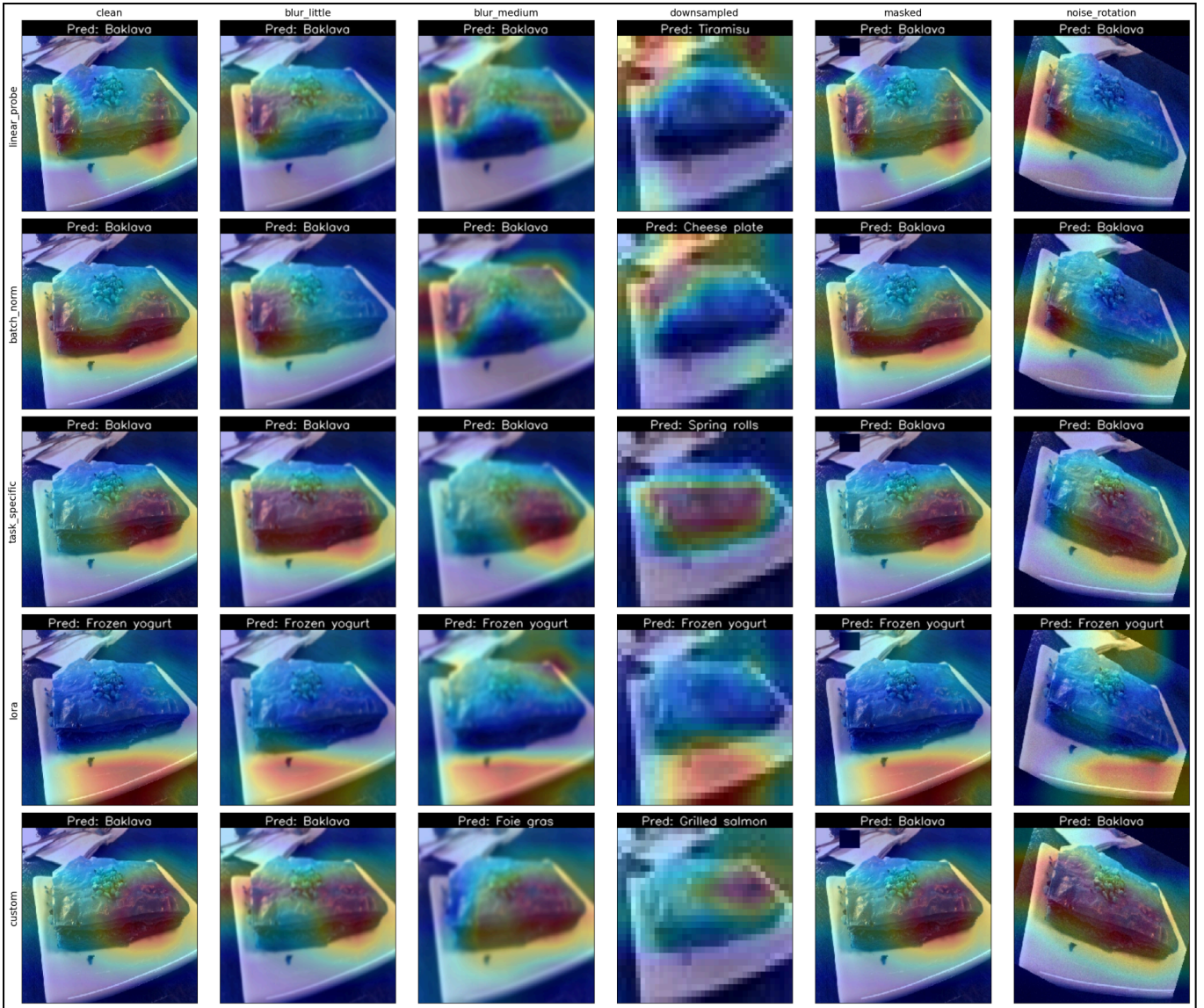


Figure 5: Grad-CAM visualization for ResNet18-based models across different fine-tuning methods and input conditions. Rows represent models (linear probing, BatchNorm tuning, adapters, LoRA, and custom model), and columns represent input corruptions (clean, blurred, downsampled, masked, and noisy images). Warmer regions indicate higher importance for prediction.

In Figure 1, each model shows different attention behavior. The linear probing model does not focus properly on the food and often looks at background areas, especially under corrupted inputs, leading to incorrect predictions. The BatchNorm tuning model improves this by focusing more on the main food region and maintaining stable attention. The task-specific adapter model also shows better focus than the baseline but becomes slightly unstable under strong distortions. The LoRA model performs inconsistently, sometimes focusing correctly but often spreading attention under noise. The custom model performs the best, consistently focusing on the key food region even under challenging conditions, showing strong robustness and a better understanding of important features.

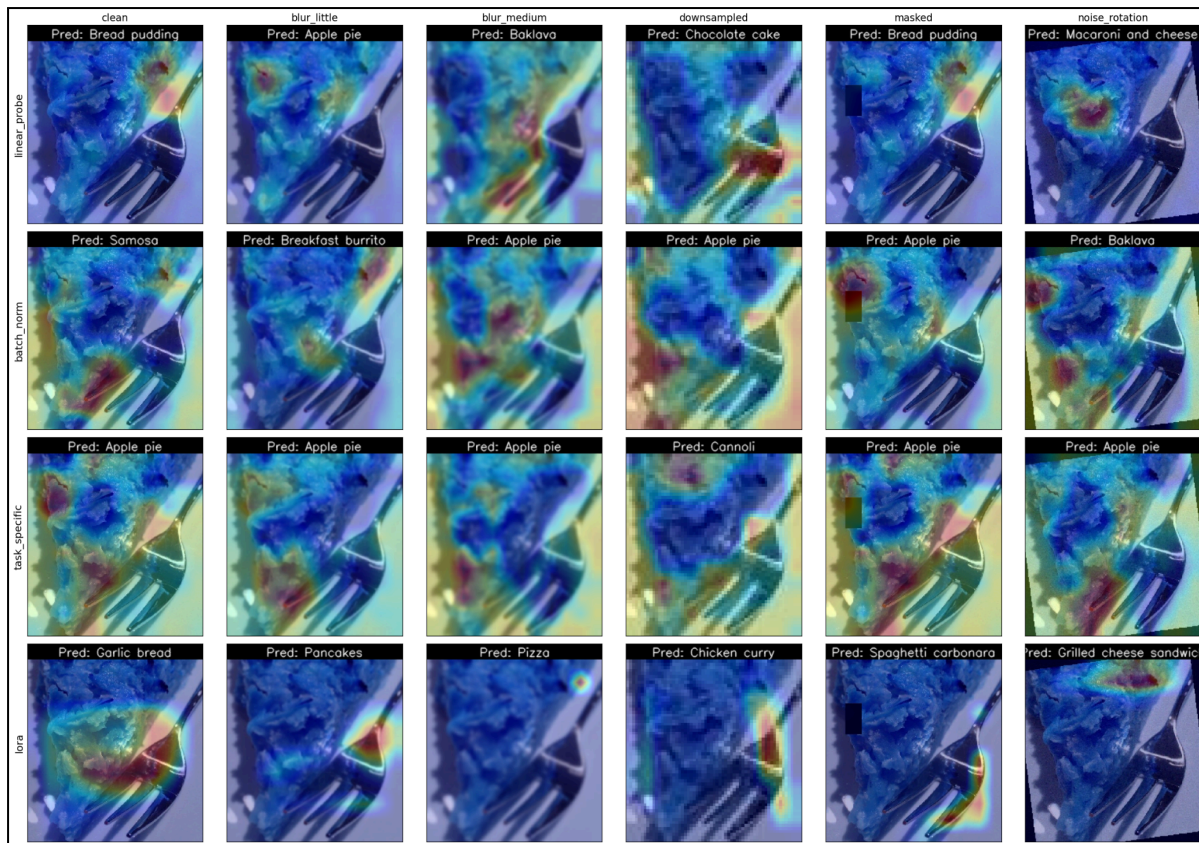


Figure 6: Grad-CAM visualization for EfficientNetV2-S models across different fine-tuning methods and input conditions.

As shown in Figure 5, EfficientNet models exhibit less stable attention compared to ResNet-based models. The linear probing baseline shows highly scattered attention, often focusing on background elements such as the fork rather than the food, resulting in incorrect predictions. While BatchNorm tuning and task-specific adapters provide slight improvements, attention remains diffuse and inconsistent under corrupted inputs. The LoRA model produces weaker activation patterns and often focuses on irrelevant regions, leading to confident but incorrect predictions. Overall, EfficientNet models demonstrate more distributed and less reliable attention, particularly under distortions, indicating lower robustness compared to the ResNet-based models.

Overall, the Grad-CAM visualizations highlight clear differences in how models utilize visual information. The linear probing baseline frequently attends to background regions, which leads to unstable predictions, especially under corrupted inputs. In contrast, BatchNorm tuning and the custom model show more consistent and concentrated focus on the main food region, indicating stronger feature alignment with the task. The adapter-based models provide moderate improvements, while LoRA exhibits less stable attention patterns under noise and distortions.

An important observation is that models with more localized and stable attention tend to maintain more reliable predictions when the input is degraded. Although EfficientNetV2-S achieves higher overall accuracy, its attention maps appear more distributed in these examples, whereas ResNet-based models exhibit more focused attention on key regions.

This suggests that robustness is influenced not only by model performance but also by how effectively relevant features are prioritized in different conditions.

5. Conclusion and Discussion for Future Work

This project studied how different fine-tuning strategies affect food classification performance and robustness on Food101. The overall conclusion is clear: parameter-efficient tuning works well, but the best method depends on the backbone and the deployment goal. The strongest overall result came from EfficientNetV2-S with BatchNorm tuning, which achieved the highest clean accuracy and the best average performance across all corrupted test sets. This model shows that a pretrained backbone can be adapted very effectively with only limited trainable updates. The best ResNet18 variant was the custom model, which combined attention-based adapters, BatchNorm tuning, and stronger data augmentation. Although it did not beat the best EfficientNet model in absolute accuracy, it was the most stable ResNet variant and showed the smallest clean-to-corrupted drop. Grad-CAM visualizations further support these results, showing that fine-tuned models focus more consistently on the food region compared to baseline models. This improved attention helps explain their stronger performance and robustness under corrupted inputs.

There are clear limitations to our work. First, the downsampled test set remained very difficult for every method, including the best-performing models. This means that the learned representations still depend strongly on fine image detail. Second, the LoRA configurations were likely not fully optimized, so those results may underestimate what LoRA can do with a more careful search over rank, target layers, and learning rate.

There are several directions for future work:

1. Include full fine-tuning when more GPU resources are available. This would provide a stronger upper bound and help quantify how close the PEFT methods are to the full-capacity solution.
2. Tune the LoRA design more carefully. Different ranks, alpha values, and insertion points could significantly change its effectiveness on convolutional backbones.
3. Train with corruption-aware augmentation. Since blur and downsampling were the largest weaknesses, future models could include multi-scale training, stronger blur augmentation, or mixed corruption training to improve robustness.
4. Expand the custom architecture. The current custom model already improved ResNet18. Future versions could test deeper attention blocks, lightweight squeeze-and-excitation modules, or hybrid CNN-transformer ideas.
5. Add a more detailed class-level error analysis. Confusion matrices and per-class robustness scores would help identify which food categories are hardest to separate and why.

In summary, the experiments show that selective fine-tuning is a strong strategy for this problem. The project demonstrates that it is possible to obtain large improvements over a linear baseline without fully retraining the network, and it highlights an important practical

lesson: the best model is not determined by accuracy alone, but by the balance among robustness, speed, and parameter efficiency.

References

- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. *Proceedings of the 38th International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2104.00298>
- Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 7132–7141.
- Woo, S., Park, J., Lee, J.-Y., & Kweon, I. S. (2018). CBAM: Convolutional block attention module. *Proceedings of the European Conference on Computer Vision (ECCV)*, 3–19. <https://arxiv.org/abs/1807.06521>
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2106.09685>
- Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., & Gelly, S. (2019). Parameter-efficient transfer learning for NLP. *Proceedings of the 36th International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/1902.00751>
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the International Conference on Machine Learning (ICML)*, 448–456.
- Bossard, L., Guillaumin, M., & Van Gool, L. (2014). Food-101 – Mining discriminative components with random forests. *Proceedings of the European Conference on Computer Vision (ECCV)*, 446–461. https://data.vision.ee.ethz.ch/cvl/datasets_extra/food-101/
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems (NeurIPS)*, 32.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 25.